AI-Optimized MBIST Test Time Reduction

Technical Document

18 October 2026

Contents

1. Executive summary **3**

2. Background: MBIST architecture and test-time contributors **3**
2.1 Architecture building blocks . . . 3
2.2 March algorithms and their cost . . . 3
2.3 Where MBIST test time goes . . . 3

3. Problem statement and test-time model **4**

4. AI/ML optimization framework overview **4**

5. Technique 1: ML-driven test scheduling under power constraints **5**
5.1 Why heuristics leave time on the table . . . 5
5.2 Formulation . . . 5
5.3 Hybrid with exact solvers . . . 5
5.4 Power model learning . . . 6
5.5 Session balancing tactics . . . 6

6. Technique 2: Adaptive March algorithm selection and pruning **6**
6.1 Element-level detection mining . . . 6
6.2 Set-cover with uncertainty . . . 6
6.3 Algorithm assignment by memory class . . . 6
6.4 Corner reduction . . . 7
6.5 Programmable controllers . . . 7

7. Technique 3: Predictive defect modeling and adaptive test flow **7**
7.1 Risk model inputs . . . 7
7.2 Model choice . . . 7
7.3 Decision policy . . . 7
7.4 Early stop and stop-on-fail . . . 7
7.5 Retention test reduction . . . 8

8. Technique 4: Controller grouping and repair analysis acceleration **8**
8.1 Controller sharing and grouping . . . 8
8.2 Learned BIRA . . . 8
8.3 Fewer repair iterations . . . 8

9. Data pipeline, features, training and deployment **8**
9.1 Data sources . . . 8
9.2 Feature engineering . . . 9
9.3 Training and validation . . . 9
9.4 Deployment . . . 9

10. Implementation flow and EDA/ATE integration **9**
10.1 ATE integration points . . . 10

11. Results, metrics and illustrative case study **10**
11.1 Example device 10
11.2 Test time by component 10
11.3 Metrics to track 10
11.4 Cost impact 11

12. Risks, quality safeguards and validation **11**
12.1 Validation protocol 11

13. Roadmap, conclusion and references **11**
13.1 Roadmap 11
13.2 Conclusion 12
13.3 References 12

1. Executive summary

An AI-optimized MBIST flow can cut memory test time by roughly 40–50% per test insertion without raising defect escapes, by combining four levers: power-aware ML scheduling, data-driven March algorithm pruning, risk-based adaptive test flows, and learned repair analysis.

On modern SoCs, embedded memories occupy 50–70% of die area and often contribute 20–40% of total test time on ATE. Every millisecond saved scales across millions of units and multiple insertions (wafer sort, final test, burn-in, system-level and in-field test).

The approach keeps the established MBIST hardware (controllers, collars, BIRA/BISR, IEEE 1687 access) and puts machine learning in the planning and decision layers: which memories test together, which algorithm elements are needed, which die get the full flow, and when a test can stop early. Every reduction is gated by quality safeguards: sampled full-flow testing, shadow-mode validation and automatic rollback when escape indicators move.

2. Background: MBIST architecture and test-time contributors

Memory Built-In Self-Test (MBIST) places pattern generation and comparison logic on-chip so memories can be tested at speed with minimal ATE vector memory.

2.1 Architecture building blocks

- **MBIST controller:** a finite-state machine or micro-programmable engine that sequences March elements, addresses and data backgrounds.
- **Memory collar / wrapper:** muxes functional and test paths, holds the comparator and fail-capture logic next to each memory.
- **BIRA (built-in redundancy analysis):** allocates spare rows and columns from captured fail data.
- **BISR and eFuse:** stores the repair signature and reloads it at power-up.
- **Access network:** IEEE 1149.1 JTAG with IEEE 1687 (IJTAG) instrument networks, often with IEEE 1500 core wrappers.
- **Power and clock control:** test-mode clock gating, sharing of controllers across memories, and session grouping.

2.2 March algorithms and their cost

Test time of a March test grows linearly with the number of addresses N. The coefficient depends on the algorithm.

Algorithm	Complexity	Main faults targeted
MATS+	5N	Stuck-at, address decoder
March C-	10N	Stuck-at, transition, coupling (idempotent, inversion)
March LR	14N	Realistic linked faults
March SS	22N	Static simple and dynamic read-destructive faults
March RAW	26N	Dynamic read-after-write faults

Each algorithm is typically run over several data backgrounds (solid, checkerboard, row/column stripe), several port configurations, and two to four voltage/temperature corners, which multiplies its base cost.

2.3 Where MBIST test time goes

Contributor	Typical share	Driver
At-speed March execution	40–55%	Largest memory per session, algorithm length, backgrounds
Retention / data-retention tests	15–30%	Pause times of tens to hundreds of milliseconds
Repair analysis, fuse program, re-test	10–20%	Fail-capture iterations, fuse burn time
Setup, IJTAG access, shifting results	5–15%	Network length, serial diagnosis readout
Corner repetition (Vmin, Vmax, hot, cold)	Multiplier ×2–4	Quality requirements per market

Shares are representative ranges for large SoCs; actual splits depend on memory count, process node and quality targets.

3. Problem statement and test-time model

The goal is to minimize total MBIST test time per device while holding fault coverage, repair yield and defect escapes (DPPM) at or better than the baseline, under power, thermal and controller-sharing constraints.

Memories are grouped into test sessions that run in parallel. A session lasts as long as its slowest member, and sessions run one after another:

$$T_{total} = T_{setup} + \sum_{s=1}^{S} \left(\max_{i \in s} \left(k_{a(i)} \cdot N_i \cdot B_i \cdot t_{clk,i} \right) + T_{ret,s} \right) + T_{repair} + T_{readout}$$

where k is the March coefficient of the algorithm assigned to memory i, N its address count, B its number of data backgrounds and port passes, t_{clk} its test clock period, and T_{ret} the retention pause in session s.

The main constraints are:

$$\sum_{i \in s} P_i \leq P_{max} \qquad \text{(global power per session)}$$

$$\sum_{i \in s,\, i \in z} P_i \leq P_{zone,max}(z) \qquad \text{(local IR-drop zone)}$$

$$FC(a(i)) \geq FC_{target}(i) \qquad \text{(fault coverage per memory)}$$

$$|s \cap C_j| \leq 1 \qquad \text{(per serial controller } C_j\text{)}$$

The problem is NP-hard: session grouping is a constrained bin-packing problem, algorithm assignment is a coverage-constrained selection problem, and repair allocation is NP-complete. Classical flows solve each with fixed heuristics at design time. AI methods add two things: better search over large solution spaces, and the use of silicon data that only exists after production starts.

4. AI/ML optimization framework overview

The framework is a closed loop: design-time models plan an optimized test program, production data from ATE feeds back to refine models, and a quality gate decides what may be deployed.

1. **Design-time planning:** a power-aware scheduler (Section 5) and a controller-grouping optimizer (Section 8) produce sessions from the memory list, floorplan and power model.
2. **Algorithm configuration:** an algorithm selector (Section 6) assigns the shortest March variant per memory class that meets the coverage target.
3. **Production execution:** the adaptive test flow (Section 7) scores each die and selects a full, standard or reduced test path, with early-stop rules.
4. **Repair:** ML-assisted BIRA (Section 8) predicts repairability early and proposes spare allocation.
5. **Feedback and learning:** fail bitmaps, parametric data and downstream returns are logged (Section 9) and used to retrain models.
6. **Quality gate:** shadow-mode comparison, sampled full testing and drift monitors (Section 12) approve or roll back each change.

Lever	ML technique	Decision it makes	Typical saving
Power-aware scheduling	Reinforcement learning, GNN + solver	Which memories share a session	15–30% of March time
Algorithm pruning	Fault-coverage learning, Bayesian analysis	Which March elements and backgrounds to run	10–35% of March time
Adaptive test flow	Gradient-boosted risk models	Which die get full vs reduced test	30–70% of retention time
Repair acceleration	Classifiers, learned allocation	Early abort, faster spare allocation	20–40% of repair time

Savings are representative ranges; they overlap and do not add linearly.

5. Technique 1: ML-driven test scheduling under power constraints

Learned scheduling packs memories into fewer, better-balanced sessions than greedy heuristics, typically cutting March execution time by 15–30% at the same peak power.

5.1 Why heuristics leave time on the table

Greedy schedulers sort memories by test length and fill sessions until the power budget is reached. They ignore local IR-drop hotspots, mixed clock domains and the fact that a single large memory sets the length of its whole session. The result is sessions where small memories finish early and power headroom sits idle.

5.2 Formulation

- **State:** remaining memories with features (size, word width, ports, clock, power per cycle, floorplan coordinates, power domain, controller ID), current session contents and remaining power headroom per zone.
- **Action:** add a memory to the current session, or close the session.
- **Reward:** negative session length plus penalties for power or zone violations; terminal reward equals negative total test time.
- **Policy:** a graph neural network over the memory graph (edges = physical adjacency, shared controller, shared power grid) with a pointer-network decoder, trained with PPO or REINFORCE.

5.3 Hybrid with exact solvers

The RL policy proposes a schedule in milliseconds. A constraint-programming or ILP solver then refines it locally (swap and move neighbourhoods) under hard power constraints. The learned policy gives a strong starting point; the solver guarantees feasibility.

5.4 Power model learning

Static per-memory power numbers are pessimistic. A regression model (gradient boosting or small MLP) trained on silicon IDD measurements and IR-drop simulation predicts actual session power from the session mix. Lower pessimism lets more memories run concurrently.

5.5 Session balancing tactics

- Pair one large memory with many small ones so the small ones fill the large one's runtime.
- Split very deep memories across address halves where the controller supports it.
- Run retention pauses in parallel with March execution of other groups.
- Raise test clock on memories whose silicon data show margin, within timing sign-off.

6. Technique 2: Adaptive March algorithm selection and pruning

Silicon fail data show that most detections come from a small subset of March elements and backgrounds; learning that subset per memory class lets the flow drop redundant operations while keeping measured coverage.

6.1 Element-level detection mining

During the learning phase, the full algorithm runs with per-element fail logging. For each memory class (compiler, size bucket, bitcell type, Vt flavour) the system builds a detection matrix: rows are failing die, columns are March elements × backgrounds × corners, and a cell is 1 if that element first detected the fail.

6.2 Set-cover with uncertainty

Selecting the shortest element subset that detects all observed fails is a weighted set-cover problem (weight = element cycles). Because rare fault types may be absent from the learning sample, the solver uses a Bayesian estimate of each element's unique detection rate and keeps any element whose upper confidence bound exceeds the escape budget.

$$\min_{x} \sum_{e} c_e x_e \quad \text{s.t.} \quad \sum_{e} d_{f,e} x_e \geq 1 \;\; \forall f, \qquad \Pr(\text{escape} \mid x) \leq \epsilon$$

6.3 Algorithm assignment by memory class

Memory class	Baseline	Learned assignment (example)	Rationale
Large single-port SRAM, mature node	March SS, 4 backgrounds	March C- + targeted read-disturb element, 2 backgrounds	Dynamic faults rare in silicon data
High-density bitcell, advanced node	March SS	March SS kept, backgrounds reduced	Dynamic faults observed; keep coverage
Two-port register files	March C- per port	Concurrent-port March, shared address sweep	Port interaction faults detected in one pass
ROM	Full read + MISR	Unchanged	Already minimal

6.4 Corner reduction

A classifier predicts, from earlier-corner results and parametric data, whether a die would fail at a later corner. Die with high confidence of passing skip redundant corners; any uncertain die runs all corners.

6.5 Programmable controllers

Pruning needs controllers that load algorithms at test time (micro-coded or programmable MBIST). Hard-wired controllers can only choose among pre-built algorithms, which limits the saving to algorithm selection rather than element-level pruning.

7. Technique 3: Predictive defect modeling and adaptive test flow

A per-die risk model routes low-risk die through a reduced MBIST flow and high-risk die through an extended one, which cuts average test time most in retention and multi-corner testing.

7.1 Risk model inputs

- Wafer-level: die (x, y) position, radial zone, wafer and lot ID, neighbouring-die fail counts.
- Parametric: Vmin search results, IDDQ, ring-oscillator speed, leakage monitors, process-control monitor data.
- Early test results: scan and early MBIST outcomes, number of repaired rows and columns.
- Equipment context: tester, probe card, temperature log.

7.2 Model choice

Gradient-boosted trees (XGBoost, LightGBM) work well on tabular test data, handle missing values and give feature importances for engineering review. A spatial model (CNN on wafer maps, or Gaussian-process smoothing) adds neighbourhood information that tabular features miss.

7.3 Decision policy

Risk tier	Share of die (example)	MBIST flow
Low	70–85%	Pruned algorithms, shortened retention pause, reduced corners
Medium	10–25%	Standard production flow
High	2–5%	Extended flow: full algorithms, all corners, longer retention
Audit sample	1–2% random	Full baseline flow regardless of score, used for escape monitoring

Thresholds are set from a cost model that weighs test-time saving against the expected cost of an escape for the target market.

7.4 Early stop and stop-on-fail

- Stop MBIST on the first unrepairable fail when no diagnosis is needed.
- Predict unrepairable die from partial fail bitmaps and abort repair loops early.
- Skip remaining corners once a die is binned out.

7.5 Retention test reduction

Retention faults correlate strongly with leakage monitors and bitcell Vmin. A model trained on these signals lets low-risk die use a shorter pause, or test retention on a representative subset of memories, while high-risk die keep the full pause.

8. Technique 4: Controller grouping and repair analysis acceleration

ML-guided controller grouping reduces serial testing and routing overhead at design time, and learned repair analysis shortens the fail-capture, allocate and re-test loop in production.

8.1 Controller sharing and grouping

One controller per memory maximizes parallelism but costs area; one controller for many memories saves area but serializes testing. The optimizer clusters memories onto controllers using:

- physical distance from the floorplan (routing and timing cost),
- clock domain and test frequency compatibility,
- power domain membership and shutdown sequencing,
- predicted test length, so each controller's total workload is balanced.

A multi-objective search (NSGA-II or Bayesian optimization over clustering parameters) returns a Pareto front of area versus test time, and designers pick a point.

8.2 Learned BIRA

Optimal spare allocation is NP-complete; hardware BIRA uses fast heuristics (must-repair first, then greedy). Two ML additions help:

- **Repairability prediction:** a classifier on partial fail bitmaps (fail counts per row, column and block, fail clustering) flags unrepairable memories after a fraction of the test, enabling early abort.
- **Allocation policy:** a model trained on optimal solutions (from offline exact solvers) ranks rows and columns to repair, improving repair rate over greedy heuristics. It can run in the ATE host for off-chip analysis, or as compact rules distilled into BIRA logic.

8.3 Fewer repair iterations

Predicting post-repair pass probability lets the flow skip the confirmation re-test for high-confidence repairs at intermediate insertions, while keeping it at the final insertion. Fuse programming can be batched across memories to reduce burn overhead.

9. Data pipeline, features, training and deployment

Model quality depends on joining design metadata, ATE results and downstream quality data at die and memory-instance level.

9.1 Data sources

Source	Format	Content
Design database	Memory list, LEF/DEF, UPF, MBIST config	Size, ports, location, power domain, controller mapping
ATE logs	STDF, tester datalogs	Pass/fail per test, per corner, test times

Source	Format	Content
MBIST diagnosis	Fail bitmaps, failing addresses per element	Element-level detection, fault signatures
Parametric	STDF parametric records	Vmin, IDDQ, ring oscillator, leakage monitors
Fab / inline	Wafer maps, PCM data	Process context per lot and wafer
Downstream	Final test, SLT, customer returns (RMA)	Escape labels for quality validation

9.2 Feature engineering

- Memory-level: N, word width, ports, bitcell type, compiler version, repair resources.
- Session-level: total and peak predicted power, spatial spread, clock mix.
- Die-level: parametric z-scores within wafer, spatial neighbour fail rates, early-test outcomes.
- Fail-signature: fails per row/column/block, clustering metrics, element of first detection.

9.3 Training and validation

- Split by lot and time, never randomly by die, so the evaluation reflects future production.
- Handle extreme class imbalance (escapes at ppm level) with cost-sensitive loss and calibrated probabilities.
- Report recall on known-bad die at the chosen threshold, not just accuracy.
- Re-train on a schedule (for example monthly) and after process or product changes.

9.4 Deployment

Scheduling and algorithm-selection models run offline and output a test program. Risk models run on the ATE host or a test-floor edge server with a latency budget of a few milliseconds per die. Models are versioned with the test program, and every decision is logged with model version and inputs for traceability.

10. Implementation flow and EDA/ATE integration

The AI layer sits around standard commercial MBIST flows (for example Siemens Tessent MemoryBIST, Synopsys STAR Memory System, Cadence Modus) and exchanges data through their configuration files, IJTAG PDL and ATE pattern formats.

1. **Extract the memory inventory** from the netlist and MBIST configuration: instances, sizes, ports, clocks, power domains, floorplan coordinates.
2. **Plan controllers and sessions** with the grouping optimizer and scheduler; write the result back as MBIST insertion constraints and session definitions.
3. **Insert and verify MBIST hardware** using the EDA tool; use programmable controllers where element-level pruning is planned.
4. **Generate patterns** for baseline and candidate algorithm sets; retarget PDL to chip level through the IEEE 1687 network.
5. **Learning phase on first silicon:** run the full baseline flow with element-level fail logging on several thousand die across lots.
6. **Train models** for algorithm pruning, risk scoring, power prediction and repairability.
7. **Shadow mode:** run the baseline flow in production while the models record what they would have done; compare decisions with actual outcomes.
8. **Staged rollout:** enable reductions on a fraction of volume, with the audit sample at full flow, then expand as escape metrics hold.
9. **Continuous monitoring and retraining** through the feedback loop in Section 9.

10.1 ATE integration points

- Test program flow branches keyed on the risk tier returned by the model service.
- Per-site parallelism preserved: risk scoring must not stall multi-site test.
- Datalogging extended to record model version, inputs and chosen path per die.
- Fallback: if the model service is unavailable, the program defaults to the full baseline flow.

11. Results, metrics and illustrative case study

In the modeled example below, MBIST time per die at final test falls from 660 ms to 340 ms, a 48% reduction, with DPPM held at baseline. The figures are an illustrative engineering model, not measured data from a specific product.

11.1 Example device

A mobile application processor on an advanced node with about 1,200 SRAM, register-file and ROM instances (roughly 96 Mbit), 64 MBIST controllers, two production corners, and a quality target of under 50 DPPM.

11.2 Test time by component

Component	Baseline (ms)	AI-optimized (ms)	Reduction	Main lever
At-speed March execution	310	185	40%	Scheduling + algorithm pruning
Retention tests	200	60	70%	Risk-based adaptive flow
Repair analysis, fuse, re-test	90	55	39%	Repairability prediction, fewer iterations
Setup and IJTAG readout	60	40	33%	Fewer sessions, compressed readout
Total	**660**	**340**	**48%**	

Retention saving is an average across tiers: low-risk die (about 80% of volume) use a short pause, while medium, high-risk and audit die keep longer or full pauses.

11.3 Metrics to track

Metric	Definition	Acceptance criterion
Test time per die	ATE time for MBIST blocks	Primary target
Throughput (UPH)	Units per hour per tester	Increase in line with time saving
Fault coverage	Modeled coverage per memory class	No drop below target
Audit escape rate	Fails found by full flow in audit sample that reduced flow would miss	Below statistical threshold
DPPM	Downstream and customer failures	At or below baseline
Repair yield	Repairable die / total die	At or above baseline
Model drift	Shift in input distributions and score calibration	Within control limits

11.4 Cost impact

At an assumed ATE cost of USD 0.03 per second, saving 320 ms per die saves about USD 0.0096 per die per insertion. At 50 million units a year that is roughly USD 480,000 per insertion per year, before capacity gains from deferred tester purchases.

12. Risks, quality safeguards and validation

The main risk is a defect escape caused by a model that under-tests a bad die; every reduction therefore needs a statistical guard and a fallback to the full flow.

Risk	Cause	Safeguard
Defect escapes	Pruned elements or skipped corners miss a fault type	Random audit sample at full flow; Bayesian escape bound per change
Rare fault types unseen in training	Learning sample too small	Keep elements with non-zero upper-bound unique detection; periodic full-flow lots
Model drift	Process shift, new fab, new tester	Input distribution monitors, score calibration checks, automatic fallback
Power violations from aggressive schedules	Power model under-predicts	Hard ILP constraints, silicon IDD validation, guardband
Traceability gaps	Decisions not logged	Log model version, inputs and path per die in STDF
Safety-critical compliance	Automotive (ISO 26262) or in-field MBIST requirements	Keep certified baseline algorithms for safety mechanisms; apply AI only where diagnostic coverage is preserved
Tester service failure	Model service down	Default to baseline program

12.1 Validation protocol

1. Offline back-testing on historical lots, split by time.
2. Shadow mode on live production for at least several weeks of volume.
3. Limited release on a fraction of volume with full downstream tracking.
4. Formal sign-off by test, quality and product engineering before full release.
5. Requalification after any process, design or tester change.

13. Roadmap, conclusion and references

13.1 Roadmap

Phase	Scope	Exit criterion
1. Foundation	Data pipeline, element-level fail logging, baseline metrics	Joined dataset across design, ATE and downstream
2. Design-time optimization	ML scheduling and controller grouping on next tape-out	Modeled March time reduced 15% or more at same power
3. Production adaptivity	Algorithm pruning and risk-based flow in shadow mode	Audit escape rate within bound over sustained volume

Phase	Scope	Exit criterion
4. Full release	Staged rollout across products and insertions	DPPM at or below baseline, target time saving met
5. Extensions	In-field and system-level MBIST, cross-product transfer learning	Models reused on new products with less learning data

13.2 Conclusion

MBIST test time is driven by decisions made with fixed rules: session grouping, algorithm choice, uniform test for every die and heuristic repair. Machine learning turns those into data-driven decisions, using design models before silicon and fail data after it. With statistical guards, audit sampling and fallbacks, a 40–50% reduction in MBIST time per insertion is a realistic target for large SoCs, while keeping coverage, repair yield and DPPM at baseline.

13.3 References

Foundational and standards literature (cited from general knowledge; confirm editions before external use):

- A. J. van de Goor, *Testing Semiconductor Memories: Theory and Practice*, Wiley, 1991.
- S. Hamdioui, A. J. van de Goor, M. Rodgers, “March SS: A Test for All Static Simple RAM Faults,” IEEE MTDT, 2002.
- IEEE Std 1687, *Access and Control of Instrumentation Embedded within a Semiconductor Device* (IJTAG).
- IEEE Std 1500, *Embedded Core Test*.
- IEEE Std 1149.1, *Test Access Port and Boundary-Scan Architecture*.
- Y. Zorian, “A Distributed BIST Control Scheme for Complex VLSI Devices,” IEEE VLSI Test Symposium, 1993 (power-constrained BIST scheduling).
- ISO 26262, *Road vehicles: Functional safety*, Part 11 (semiconductor guidance).